

Algorithm for file updates in Python

Project description

In this project, an algorithm was developed to review users that are authorized to access restricted personal patient records, as well as remove any users that no longer have access to this content. This was done by comparing the previous list of authorized IP addresses to the IP addresses that needed to be removed. Once compared, the algorithm would remove any IP addresses that appeared on both the allowed IP address list and the remove list.

Open the file that contains the allow list

The first step was opening the file that contained the allow list. I started by using the `with` statement followed by the `open()` method to open the file containing the allowed IP addresses. This `allow_list.txt` was stored within the variable `import_file`. I then stored the file that I was opening under the variable `file`.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

Once the file was opened, it then needed to be converted into a string that would list out the contents of the file. To do this, the `read()` method was appended to the file variable that was established in the open statement. The output of that was saved to a variable named `ip_addresses` and the contents of that were then displayed using the `print()` function

```
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Convert the string into a list

Next I needed to convert the string into a list to begin parsing through the data and removing the individual IP addresses from the allow list. This was done using the `.split()` method. The `.split()` keyword was appended to the `ip_addresses` variable and the result of that was then renamed `ip_addresses`

```
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
# Display `ip_addresses`
print(ip_addresses)
```

Iterate through the remove list

The next step was to see if the IP addresses that were in the remove list were present in the allowed IP addresses file, and if they were, they needed to be removed. In order to do this, a **for** loop was utilized to iterate through each element (stored under the loop variable name *element*) in the `ip_address` variable's list. The result of this for loop was then printed to display each element iterated out line by line

```
ip_addresses = ip_addresses.split()
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)
```

Remove IP addresses that are on the remove list

Next, an **if** statement was placed to set the condition stating if *element* is **in** the *remove_list* as well, then that IP address would need to be removed, using the `.remove()` method. The result was then printed to display the remaining ip addresses

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)

['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9', '192.168.52.90', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.69.116']
```

Update the file with the revised list of IP addresses

Now that the ip addresses have been removed from the *ip_address* variable. I had to complete the algorithm by updating the file with the revised list. To do this, the list of IP addresses had to be converted back into a single string. This was done using the `.join()` method. The `.join()` keyword was added to the empty string "", which signified that each new string would be identified by white space. The new *ip_addresses* variable stored the `.join(ip_addresses)` code

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)
```

Lastly I had to use the **with** statement to open up the import file and update the old allowed list of IP addresses by replacing it with the new *ip_addresses* list. The `.write()` method was used for this

```
# Build `with` statement to rewrite the original file

with open(import_file,"w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

Summary

To summarize I created an algorithm that was put in place to update an old list of authorized users to a more up to date version. I was able to do this by instituting code that imported and

read the contents of a file, converted it into a list that allowed me to create an iteration with conditions stating that if the IP addresses to be removed were also in the approved IP address list, those IP addresses would be removed. Once this was updated, the list was converted back into a string and the original file's contents were replaced with that of the newly updated IP addresses.